

FoundIt — Live Demo Script

Team & roles

Person	Role in the story
Nidhi	Projected screen, narration. Loses the notebook, claims it back.
Shenza	Second visible screen. Finds the notebook, posts it, approves the claim.
Shanid	AI assistant search, flagging / moderation.
Hadi	Marketplace buyer.

Target length: 9-11 minutes + questions **URL:** <https://found-it-ashy-sigma.vercel.app>

Steps marked **[BLAZE]** need Cloud Functions deployed. Without them the action appears to succeed but nothing happens server-side. Decide before you present which version you are running and delete the other branch from your notes.

0. Pre-flight (do this 30 minutes before, not in the room)

- ☐ All four devices signed out, popups allowed for the site on every one (Chrome blocks the Google window by default — this is the single most likely thing to break the demo)
- ☐ `found-it-ashy-sigma.vercel.app` open in one tab per device, hard-reloaded
- ☐ Nidhi's screen mirrored to the projector; Shenza's device visible to the room too if possible — the side-by-side is what sells real-time sync
- ☐ Feed shows 14 seeded items, not an empty list
- ☐ One rehearsal of the full run on the actual room wifi
- ☐ Firestore rules deployed (`users` read must return 403 — see below)
- ☐ Phone hotspot ready as a fallback if campus wifi is hostile

Rules check:

```
curl -s -o /dev/null -w "%{http_code}\n" \  
  "https://firestore.googleapis.com/v1/projects/foundit-  
  fcfcc/databases/(default)/documents/users?pageSize=1"
```

`403` = correct. `200` = every signed-in student's campus email is public.

1. The problem (Nidhi, 45s) — no clicking yet

"Every semester our campus loses hundreds of items — ID cards, calculators, water bottles — and the recovery process is a WhatsApp group with 300 people and no memory. Separately, seniors leaving campus sell furniture and cycles through the same chaotic channel. FoundIt merges both into one trusted, verified-student-only platform."

Land the architectural framing early — the course is Software Architecture, not web development:

"The interesting problem here isn't the UI. It's that lost-and-found is event-driven and needs server-authoritative trust. We'll show why."

2. The feed + campus-only auth (Nidhi, 90s)

1. Show the populated feed. Point out the three card types: lost, found, marketplace.
2. Use the category and location filters. Use the search box.
3. Click **Sign in** → **Continue with Google**.
4. While the Google window is open, say:

"Only @iitkottayam.ac.in accounts get in. Notice we pass Google a domain hint — but that's just a UI filter the client controls, so we re-check the domain server-side after sign-in and sign the session straight back out if it doesn't match. Client-side checks are suggestions, not security."

5. Signed in. Show the avatar / account menu.

If the popup is blocked: the app now tells you exactly what to do. Click the blocked-popup icon in the address bar, allow, retry. Don't improvise.

3. Real-time sync — the strongest moment (Nidhi + Shenza, 2 min)

This one works with or without Blaze. Make it the centrepiece.

1. **Shenza** signs in on device 2, feed visible to the room.
2. **Nidhi** clicks **Post**, files a *lost* report:
 - Title: Lost blue spiral lab record notebook
 - Category: **Books & Notes** · Location: Lecture Halls (LH)
 - Description: Blue spiral-bound lab record, name on the inside cover, last used in the LH-101 electronics lab.
3. Submit. **Do not reload Shenza's device.**

Why a notebook and not a calculator. The seeded feed already contains a lost and a found Casio calculator, plus paired water bottles and ID cards. Posting another calculator would put four near-identical cards on screen and, worse, Nidhi's new item would match the seeded found calculator as well as Shenza's — so the match badge could point at the wrong card mid-demo. Books & Notes has only marketplace listings seeded, no lost/found, so this pair is guaranteed to match each other and nothing else.

"No refresh. Firestore pushes the change over an open snapshot listener — every connected client is a subscriber. This is the Observer pattern, and it's why we chose a realtime document store over a REST API."

4. Let the room see the card appear on both screens.

4. Smart matching (Shenza, 90s) [BLAZE]

1. **Shenza** posts the complementary *found* report — she is the finder, Nidhi is the loser; the function needs one `lost` and one `found` doc from *different* posters (it skips same-poster candidates):
 - Title: Found blue spiral lab record book in LH-101
 - Category: **Books & Notes** · Location: Lecture Halls (LH)
 - Description: Picked up a blue spiral lab record after the electronics lab, handed to the department office.
 - Keep the words *blue*, *spiral*, *lab*, *record* in both posts — keyword overlap is the heaviest factor in the match score
2. Within a few seconds both cards show a match badge; Nidhi's bell shows a notification.

"Neither client computed that. A Cloud Function fires on the document write, scores every candidate of the opposite type on five weighted factors — keyword overlap by Jaccard coefficient, category, zone, time proximity, reporter trust — and writes the result back. The clients can't write `matchScore`; our security rules reject it. Server-authoritative by design."

Without Blaze: skip the live post. Instead open the seeded ID-card pair, show the 91% badge, and say plainly: "These scores come from our matching function, which we ran against the Firebase emulator suite — we haven't enabled billing on the project, so it isn't live in this deployment." Do not pretend it computed live. If a grader asks, offer to show the emulator run.

5. Claim + verification (Nidhi claims, Shenza approves, 90s)

Nidhi lost the notebook, so Nidhi is the one who claims it back. Shenza posted the found report, so Shenza is the finder who approves. Keep those roles straight — a grader watching the wrong person approve their own claim will ask.

1. **Nidhi** opens the *found* lab record Shenza posted, clicks **Claim**.
2. Answers the proof question — something only the real owner knows (`name written inside the front cover`) — picks a meeting spot, submits.
3. **Shenza** opens the claim on her device.

"The finder never sees a claimant's answer until they've submitted their own — that's the anti-fraud step. Approval is a two-party handshake, not a button one person presses."

4. **[BLAZE]** Shenza approves → item flips to *resolved*, both parties get notified, her `resolvedCount` increments and trust recomputes.

Without Blaze: the claim submits and is visible, but approval won't resolve the item. Stop after step 3 and describe the rest.

6. Marketplace deal (Hadi, 60s) [BLAZE]

1. **Hadi** opens the Trek bicycle listing, makes an offer.
2. Seller and buyer each confirm.
3. Listing flips to *sold*, review unlocks for both.

"Either side alone can't mark it sold. Both confirmations are recorded in a Firestore transaction and only the function writes the final status — the Saga pattern applied to a two-party exchange."

Without Blaze: this throws. **Skip it entirely** — don't click confirm and hope. Describe the handshake over the listing screen instead.

7. AI assistant (Shanid, 60s) — works regardless

1. Open **AI Search**, type a natural-language query: `i lost my id card`
2. Show that it returns the ID-card items, not random ones.

"Two paths behind one interface. With a Gemini key we send the query plus the candidate set to the model. Without it — or if the API fails — we fall back to a local ranked keyword search. Strategy pattern, and the feature degrades instead of disappearing."

Worth mentioning if you have 15 spare seconds — it's a genuine engineering story:

"Our first version returned a water bottle for 'I lost my ID card'. The word lost was matching the item's `type` field, and the tokenizer dropped 'id' for being two characters. We now strip intent words and require word-boundary matches on short tokens."

8. Trust + moderation (Nidhi, 45s)

Open any profile or point at the trust badges.

"Trust is Bayesian with damping: $(\text{avg} - 3) \times (\text{count} / (\text{count} + 5)) \times 40$. One five-star review from a friend barely moves you; consistent behaviour over many exchanges does. It's computed only in Cloud Functions, and the rules use `keeps()` guards so a client physically cannot write its own `trustScore`."

Flagging: three reports auto-hide content pending moderator review — Chain of Responsibility. **[BLAZE]** to show live.

9. Architecture close (Nidhi, 60s)

Put the diagram up.

"Event-driven core, CQRS-ish read model with denormalized feed documents, five security layers with the rules as the gateway, and graceful degradation at every external dependency — AI, weather, images. Twelve patterns, all in the document."

Then stop talking. Let them ask.

If something breaks

Symptom	Do this
Popup blocked	Address-bar icon → allow → retry. Scripted, not a surprise.
Feed empty	Hard reload. Mock data shows only when Firestore returns nothing.
Sign-in loops back signed out	Wrong Google account — must be the campus one.
A device is offline	Carry on with the others. Never debug live.
Deal confirm errors	You're on the no-Blaze path. Skip, describe it.

Golden rule: if something fails, say what *should* happen, move on, and offer to show it after. A confident explanation beats a panicked retry every time.

Likely questions

- *Why Firestore over SQL?* Real-time subscriptions, offline cache, and rules as a declarative authorization layer. Cost is denormalization and no joins.
- *How do you stop someone inflating their own trust?* Clients can't write it. Rules `keeps()` those fields; only the Admin SDK in functions bypasses rules.
- *What if Gemini is down?* Local keyword ranking. Demonstrated fallback.
- *Is the `hd` parameter security?* No — a client-controlled UI hint. We re-check the domain after sign-in and sign out mismatches.
- *Why aren't functions deployed?* v2 requires the Blaze plan. Tested against the emulator suite; 16 assertions across all eight functions.
- *Biggest architectural regret?* The 700 kB bundle — no code splitting. Route and modal level splitting is the obvious next step.